

II.1102 – Algorithmique et Programmation

Examen écrit

Consignes générales :

- La durée de cet examen est de **3 heures**.
- Prenez le temps de lire l'énoncé dans son intégralité et assurez vous que le sujet comporte bien 4 pages.
- Le sujet est à rendre avec la copie.
- Aucun document **n'est autorisé** pour cet examen.
- Ce partiel est noté sur **40**.
- Attention à ne pas confondre **afficher** et **retourner**.
- Veuillez faire attention à la syntaxe utilisée en **Java**. Quelques erreurs pourront être tolérées mais un oubli systématique des règles de syntaxe sera pénalisé.
- Vous pouvez rédiger votre réponse en **français** ou en **anglais**.

1. Questions de compréhension : (8 Points)

- Que signifient les modificateurs **private**, **protected**, **public** et **"default"** ?
- À quoi servent les accesseurs (**getters**) et mutateurs (**setters**) ?
- Quelle est la différence entre une **interface** et une **classe abstraite** ?
- Quel est l'intérêt des dictionnaires (**Map**) par rapport aux autres structures de données. Proposez un **cas d'usage d'utilisation** des Maps.

2. Compilation et correction du code (9 Points)

2.1. Lisez attentivement le programme ci-après :

```
import java.util.ArrayList;
public class CompilationArrayList {
    public static void main(String[] args) {
        ArrayList<Integer> nombres = new ArrayList<Integer>();
        nombres.add(1);
        nombres.add(2);
        nombres.add(3);
        nombres.add(4);
        nombres.add(5);
        int total = sommePairs(nombres, nombres.size());
        System.out.println("Somme des nombres pairs : " + total);
    }
    public static int sommePairs(ArrayList<Integer> liste, int index)
    {
        if (index <= 0) { return 0; }
        if (liste.get(index) % 2 == 0) {
            return liste.get(index) + sommePairs(liste, index - 1);
        } else { return sommePairs(liste, index - 1); }
    }
}
```

- Quel est le **résultat affiché** en console par ce programme ?

2.2. Soit le programme suivant :

```

1 public class ExerciceErreurs {
2
3     public static void main(String args) {
4         int nombre = "10";
5         system.out.println("Le nombre est : " + nombre);
6
7         for (int i = 0; i <= 5; i++);
8             System.out.println("i vaut : " + i);
9
10        String message;
11        if(message.equals("Bonjour"))
12            System.out.println("Message reçu !");
13
14        int[] tableau = new int[5];
15        tableau[5] = 100;
16
17        afficherMessage();
18    }
19
20    public void afficherMessage() {
21        System.out.println("Ceci est un message.");
22    }
23 }

```

TAF : Ce programme contient 8 erreurs, citez-les sur votre copie. **Pour chaque erreur, mentionnez le numéro de la ligne et la correction à envisager.**

3. Exercices de programmation (16 Points)

3.1. Chaînes de caractères itératif / Récursif

Nous voulons écrire un programme qui supprime **toutes les occurrences d'un caractère donné** dans une chaîne. Vous allez proposer deux méthodes :

- Une méthode **itérative**.
- Une méthode **réursive**.

Exemple :

Entrée : "programmation", **caractère à supprimer** = 'm'

Sortie : "prograation"

1. Écrire un programme Java **itératif** qui supprime **toutes les occurrences d'un caractère donné** dans une chaîne. Calculez la **complexité de ce programme**.
2. Écrire un programme Java **récuratif** qui supprime **toutes les occurrences d'un caractère donné** dans une chaîne. Calculez la **complexité de ce programme**.

3.2. Tri par sélection

Vous devez écrire une fonction qui trie un tableau d'entiers dans l'ordre croissant en utilisant l'algorithme du tri par sélection.

Le tri par sélection est un algorithme de tri simple et intuitif, qui consiste à **sélectionner à chaque étape le plus petit élément** de la partie non triée du tableau, puis à l'échanger avec l'élément au début de cette partie.

Déroulement du Tri par sélection

Pour un tableau de n éléments :

1. **Parcourir** le tableau à partir de l'indice $i = 0$ jusqu'à $n-2$.
 2. À chaque position i , **trouver l'indice du plus petit élément** entre i et $n-1$.
 3. **Échanger** ce plus petit élément avec l'élément à l'indice i .
 4. Répéter jusqu'à ce que toute la partie non triée soit triée.
-
1. **Ecrire un programme** réalisant le tri par sélection.
 2. Quelle est la **complexité** en nombre de comparaisons et d'opérations de ce programme?

4. Modélisation UML (7 Points)

Créez un diagramme de classes en vous basant sur la description suivante:

Vous êtes chargé de modéliser le système d'information d'une compagnie aérienne. Le système doit permettre de :

- Gérer plusieurs **vols** opérés par une **compagnie aérienne**.
- Chaque **vol** a un **numéro**, une **date de départ**, une **heure de départ**, un **aéroport de départ** et un **aéroport d'arrivée**.
- Chaque **vol** est associé à un **avion**.
- Un **avion** a un **code**, un **type**, et une **capacité** (nombre de places).
- Plusieurs **passagers** peuvent être associés à un vol.
- Un **passager** a un **nom**, un **prénom** et il est identifié par un **numéro de passeport**.
- Une **réservation** contient le **passager**, le **vol**, une **classe** (économique ou business), et un **statut** (confirmée, en attente, annulée).

Travail demandé : Etablir le **diagramme de Classes** pour ce cas d'usage.